TriCore™ AURIX™ Family

32-bit

MC-ISAR Safety Information

Infineon Software Architecture

User Manual

Draft v3.1 2017-02

Microcontrollers

Edition 2017-02

Published by Infineon Technologies AG,

81726 Munich, Germany.

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS APPLICATION NOTE IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS APPLICATION NOTE MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS APPLICATION NOTE.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (**www.infineon.com**).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

Confidential

Revision History

Date	Version	Changes
2017-02-16	3.1	1. Safety Objective chapter added 2. Safety Module Groups and Assumption removed 3. New AoUs are added. i.e., requirement ID SM_AURIX_MCAL_096 to SM_AURIX_MCAL_116 are added 4. AoUs SM_AURIX_MCAL_002, SM_AURIX_MCAL_004, SM_AURIX_MCAL_005, SM_AURIX_MCAL_010, SM_AURIX_MCAL_016, SM_AURIX_MCAL_018, SM_AURIX_MCAL_019, SM_AURIX_MCAL_020, SM_AURIX_MCAL_024, SM_AURIX_MCAL_030, SM_AURIX_MCAL_031, SM_AURIX_MCAL_032, SM_AURIX_MCAL_039, SM_AURIX_MCAL_050, SM_AURIX_MCAL_064, SM_AURIX_MCAL_068, SM_AURIX_MCAL_078, SM_AURIX_MCAL_084, SM_AURIX_MCAL_090 are removed, since these AoUs are not relevant for MCAL. 5. AoUs SM_AURIX_MCAL_014, SM_AURIX_MCAL_017, SM_AURIX_MCAL_067, SM_AURIX_MCAL_091 are modified
2016-04-01	3.0	Reformulated the assumed requirements based on review comments Term integrator used instead of user
2016-03-03	2.1	1. Updated Abbreviations and References 2. Added Software architecture assumptions in chapter 2 3. Additional requirement to user added, to use the supported safety mechanisms of MCAL in chapter 3 4. Common and Module specific AoUs are added in chapter 4 & 5 5. Rationale added for AoUs 6. Unique identifier for each AoU added
2015-11-05	2.0	Updated based on review points REV_008334
2015-08-05	1.9	Added chapter 6 to update as per the latest AURIX Safety Manual V1.2 and internal review points.
2015-07-23	1.8	Updated based on review points: REV_008192.
2015-07-21	1.7	Updated based on 0000051124-146: Module names are changed to have consistency with release notes, reqtify tags syntax corrected.
2015-06-10	1.6	PORT and DIO is added in chapter 5
2015-04-30	1.5	Review comments (REV_007628) incorporated
2015-04-27	1.4	Include Assumption of use for PWM in section 5.7 Elaborate section 4.8
2014-09-26	1.4	Confidential Status changed, wrong footer corrected
2014-09-10	1.3	Review comments (REV_006186) incorporated, added more explanations in 5.1
2014-09-08	1.2	Tag added in Sec 5.1(AI00248867)
2014-08-29	1.1	Review comment in section 4.4 and 4.6 incorporated.
2014-08-21	1.0	Review comments incorporation REV_006043
2014-08-12	0.8	Added CoU and changed the chapter headings, traceability tags
2014-08-11	0.7	Complete template revamped and organization of information changed
2014-05-20	0.6	Safe MCAL common functions are added in section 3.3.4.

Confidential

2014-03-25	0.5	File name changed and made generic to EP/HE.
2014-01-28	0.4	REV_005182 – review comments incorporated
2014-01-27	0.3	237795, 237796 UTP update
2013-10-11	0.2	Condition of Use section added.
2013-08-22	0.1	Initial version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing? Your feedback will help us to continuously improve the quality of our documentation. Please send your proposal (including a reference to this document title/number) to:

ctdd@infineon.com

Table of Contents

1 About this document

1.1 Purpose

This document provides the information regarding safety concept for AURIX™ MCAL software, developed as software Safety Element out of Context (SEooC), according to the ISO26262 standard. This document addresses the safety objectives, architectural assumptions and the AoUs to be fulfilled by the integrator for the usage of MCAL drivers in safety relevant applications.

1.2 Scope

[SM_AURIX_MCAL_001]This document is applicable to all devices (TC29X, TC27X, TC26X, TC23X and TC22X) in the TriCore™ AURIX™ product family. Refer **Table 2** for the targeted ASIL level of AURIX MCAL drivers. [/req]

This document does not cover the hardware safety concept. Complete understanding of the AURIX Safety Manual **[7]** is necessary before using the MCAL safety information document.

1.3 Intended Audience

Integrators who need to use AURIX™ AUTOSAR MCAL modules in safety relevant applications.

Note: The integrator shall be aware that the safety claim is not available currently for the product due to open FS issues.

1.4 Abbreviations

Table 1 Abbreviations definitions

Abbreviation	Definition
ADC	Analog to Digital Converter
AoU	Assumption of Use
API	Application Programming Interface
ASIL	Automotive Safety Integrity Level
ASW	AUTOSAR Software
ATOM	Advanced Routing Unit connected Timer Output Module
AURIX™	Automotive Real-time Integrated NeXt Generation Architecture
AUTOSAR	AUTomotive Open System Architecture
BFX	Bit field Functions for fixed point
Black Channel	The black-channel concept allows communicating safety relevant data between the sender and receiver using a potentially faulty channel (QM) – referred to as Black channel. This channel by itself does not have any data integrity measures implemented and has the potential to introduce both systematic and random failures. However, the sender and the receiver modules incorporate safety layer which can perform safety checks to detect any potential faults that has an impact on the integrity of the safety data
BSW	Basic ASW
CAN	Controller Area Network
CANTrcv	CAN Transceiver
CCU6	Capture Compare Unit 6
CRC	Cyclic Redundancy Check

Confidential

DEM	Diagnostic Event Manager
DIO	Digital Input Output
DMA	Direct Memory Access
DSADC	Delta Sigma Analog to Digital Converter
DTI	Diagnostic Time Interval
E2E	End to End
ECU	Electronic Control Unit
EMUX	External Multiplexer
ETH	Ethernet
FEE	Flash EEPROM Emulation
FLEXRAY	Flex Ray
FLS	Flash
FLSLoader	Flash Loader
FS	Functional Safety
GPT	General Purpose Timer
GTM	Generic Timer Module
HSM	Hardware Security Module
HSSL	High Speed Serial Link
HW	Hardware
I2C	Inter-Integrated Circuit
ICU	Input Capture Unit
IOM	Input Output Monitor
IRQ	Interrupt Request
LIN	Local Interconnect Network
MCAL	Micro Controller Abstraction Layer
MCALLIB	MCAL Library
MC-ISAR	Microcontroller Infineon Software Architecture
MCU	Micro Controller Unit
MPU	Memory Protection Unit
MSC	Micro Second Channel
OS	Operating System
PORT	General Purpose Input/output port line
PWM	Pulse Width Modulation
RTE	Run Time Environment
SafeTlib	Safety Test Library
SENT	Single Edge Nibble Transmission
SEooC	Safety Element Out Of Context
SFR	Special Function Register

Confidential

SPI	Serial Peripheral Interface
STDLIN	Standard Local Interconnect Network
STM	System Timer
SW	Software
TIM	Timer Input Module
TOM	Timer Output Module
UART	Universal Asynchronous Receiver/Transmitter
WDG	Watchdog

1.5 References

[1] Target TC27xB user manual: tc27x_um_v1.4.1.pdf
[2] Target TC27xC user manual: tc27xC_um_v2.2.pdf
[3] Target TC27xD user manual: tc27xD_um_v2.2.pdf
[4] Target TC26xB user manual: tc26xB_um_V1.3.pdf
[5] Target TC29xB user manual: tc29xB_um_V1.3.pdf
[6] Target TC23x/TC22x user manual: tc23x_tc22x_um_v1.1.pdf
[7] AURIX Safety Manual, AP32224, Application Note, V1.4, 2015-11

2 AURIX™ SEooC AUTOSAR Software Architecture

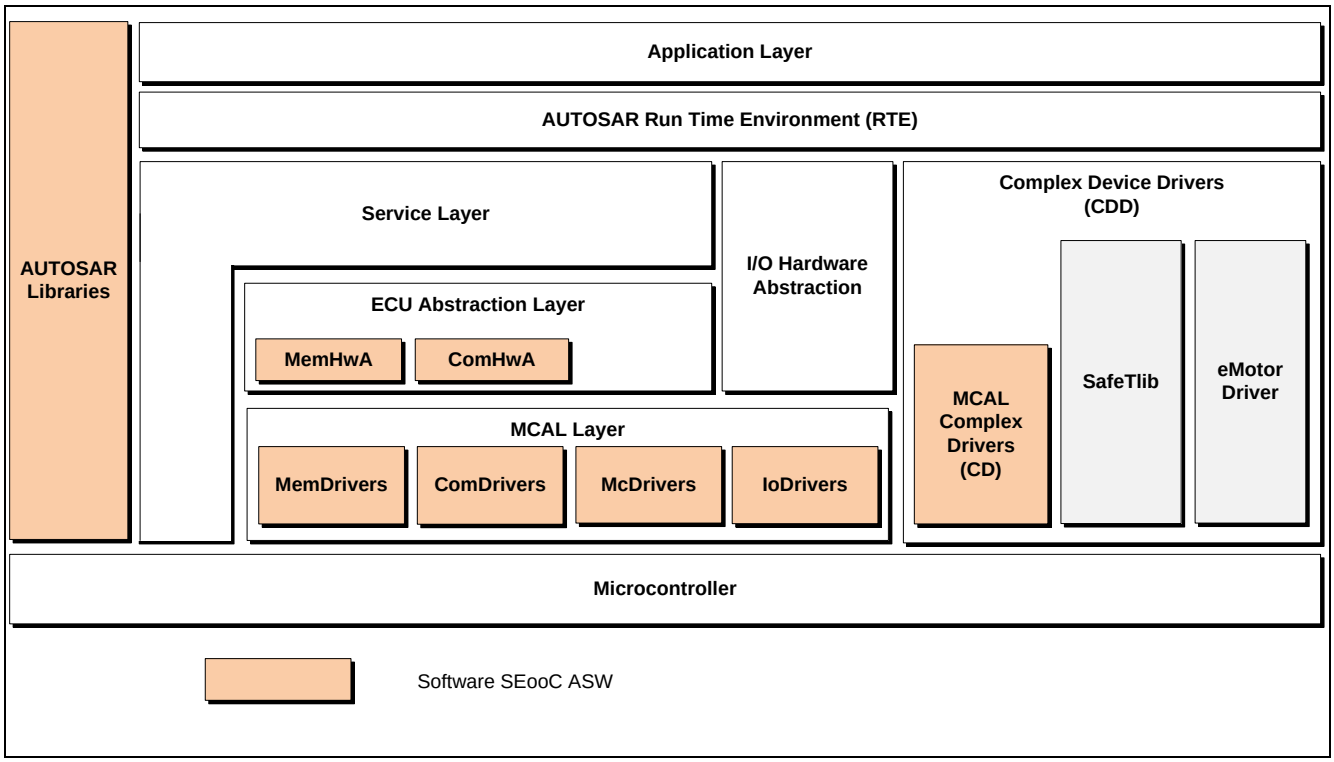

Figure 1 Software Architecture of AURIX™ SEooC ASW

The **Figure 1** shows how the software packages, for example, IoDrivers are located in ECU software architecture for a single-core system based on AUTOSAR Basic Software. From a functional point of view, the

software packages are independent of each other, as in the MCAL layer there are no horizontal interfaces between MCAL modules. All API's will be triggered from user software.

2.1 Safety Objectives

AURIX MCAL software is developed as a software safety element out of context (SW SEooC). The top level safety objective is to develop the AUTOSAR based MCAL modules in compliance with ASIL B process in order to enable them to be used in safety relevant use cases.

The drivers to be supported are

1. **Microcontroller Drivers (McDrivers)**
   - Microcontroller Unit (MCU) Driver including GTM initialization
   - Watchdog (WDG) Driver for the internal CPU watchdogs
   - General Purpose Timer (GPT) Driver
2. **Memory, Memory Hardware Abstraction Drivers (MemDrivers)**
   - Internal Flash (FLS) Driver
   - Flash EEPROM Emulation (FEE) Driver
3. **I/O Drivers**
   - Port (PORT) Driver
   - Digital I/O (DIO) Driver
   - Analog to Digital Converter (ADC) Driver
   - Pulse Width Modulation (PWM) Driver
   - Input Capture Unit (ICU) Driver
4. **Communication Drivers, Communication Hardware Abstraction Drivers (ComDrivers)**
   - Controller Area Network (CAN) Driver
   - Local Interconnect Network (LIN) Driver
   - Flexray (FR) Driver
   - Serial Peripheral Interface (SPI) Handler Driver
   - Ethernet (ETH) Driver
   - CAN Transceiver (CANTRCV) Driver
5. **AUTOSAR Libraries**
   - CRC Library

Table 2 gives the targeted ASIL level of AURIX MCAL drivers. This ASIL level only indicates the process followed for the development of the mentioned MCAL drivers.

Table 2 ASIL levels for AURIX MCAL drivers

Sl No	Driver name	ASIL level	Driver Group
1	MCU	ASIL B*	McDrivers
2	GPT	ASIL B*	McDrivers
3	PORT	ASIL B*	I/ODrivers
4	DIO	ASIL B*	I/ODrivers
5	ADC	ASIL B*	I/ODrivers
6	PWM	ASIL B*	I/ODrivers
7	ICU	ASIL B*	I/ODrivers
8	SPI	ASIL B*	I/ODrivers
9	CRC	ASIL B*	AUTOSAR Libraries
10	MCALLIB	ASIL B*	MCAL Libraries

11	WDG	QM	McDrivers
12	BFX	QM	AUTOSAR Libraries
13	CAN	QM	ComDrivers
14	LIN	QM	ComDrivers
15	FLEXRAY	QM	ComDrivers
16	ETH	QM	ComDrivers
17	FLS	QM	MemDrivers
18	FEE	QM	MemDrivers
19	CANTrcv	QM	ComDrivers
20	FLSLoader	QM	Complex Drivers
21	DMA	QM	Complex Drivers
22	UART	QM	Complex Drivers
23	MSC	QM	Complex Drivers
24	GTM	QM	Complex Drivers
25	STDLIN	QM	Complex Drivers

Refer.the Note in chapter* **1.3

2.2 Software Architecture Assumptions

[SM_AURIX_MCAL_017]The MCU, GPT, PORT, DIO, ADC, PWM, ICU, CRC, SPI drivers are intended to be developed according to ASIL B process. It is assumed that the integrator develops a safety concept for the intended safety application before using these drivers. [/req]

[SM_AURIX_MCAL_014]The CAN, CANTrcv, LIN, ETH, FLEXRAY drivers are developed according QM process. In case the integrator uses these drivers in safety application, the integrator shall perform ASIL decomposition at the system level (for example, using concepts like black channel). In this case ASIL decomposition is performed between the transmission of data via the communication stack (QM) and an associated safety mechanism (for example, the AUTOSAR E2E protection).[/req]

[SM_AURIX_MCAL_015]FEE and FLS drivers are developed according QM process. In case the integrator uses these drivers in safety application, the integrator shall perform ASIL decomposition at the system level (e.g., using concepts like black channel). In this case ASIL decomposition is performed between storage, retrieval of data via black channel which includes MEM drivers (QM) and associated safety mechanism (example. safety checks of the NVRAM Manager). [/req]

The WDG driver requirements are developed according to a QM process. In case the integrator uses WDG driver in safety application, the integrator shall perform ASIL decomposition at the system level (for example, using concepts like black channel). The WDG driver is part of the black channel (QM) and the associated safety mechanism, for example. Safety check (password verification) is performed by the CPU watchdog hardware.

[SM_AURIX_MCAL_003]It is assumed that there are no additional requirements for AURIX™ SEooC ASW when multiple cores are used, compared to the implementation for a single core.[/req]

[SM_AURIX_MCAL_007]It is assumed that the system which integrates SEooC is fail-safe and the availability of a safe state is ensured by the integrator.[/req]

[SM_AURIX_MCAL_008]It is assumed that the integrated system provides error handling (for example, safety-related SW-Cs or DEM).[/req]

[SM_AURIX_MCAL_009]It is assumed that the caller of SEooC ASW API functions in safety context has the same or higher level of ASIL capability as the one of the called ASW modules and functions. [/req]

[SM_AURIX_MCAL_011]It is assumed that the integrator uses the MPU-based partitioning, allowing to:

- Configure for each OS-Task, a sufficient number of regions for accessing RAM and memory-mapped devices

- Switch the configuration efficiently (access rights to RAM and peripherals) at runtime
- Make these restrictions both for user-mode and supervisor-mode tasks[/req]

[SM_AURIX_MCAL_012]It is assumed that the integrated AUTOSAR stack shall provide a safety-qualified OS with:

- Safe context switch between OS-Tasks
- OS-Task level partitioning (ensuring independence between OS-Tasks)[/req]

[SM_AURIX_MCAL_013]It is assumed that the hardware resources (SFR's) controlled by SEooC ASW are not used by the integrator software. [/req]

[SM_AURIX_MCAL_096]It is assumed that the invoked OS calls are developed by integrator according to ISO26262 based on the ASIL level of the intended application.[/req]

[SM_AURIX_MCAL_097]If Flash, SRAM ECC alarm (uncorrectable), is triggered application shall go to safe state and not rely on any MCAL data (for example, ADC conversion result). Refer SM_AURIX_CPU_3 and SM_AURIX_PMU_1 in HW Safety Manual **[7]** for more information.[/req]

Note: The integrator is responsible for verification of the software partitioning during software integration and testing (in accordance with Clause 10) including verification of decompositions and coexistence based on the integrator's Safety Concept and integrator's Safety Analysis.

3 Safety Mechanisms for MCAL drivers

This section provides information on the safety mechanisms against systematic software faults implemented in MCAL drivers.

1) Safety mechanisms against SW fault: Range Check for Input and Output parameters
2) Freedom from interference with respect to memory
3) All additional checks and AoU's derived out of safety criticality analysis of individual driver are detailed in chapter 5.

Note: <Mod>SafetyEnable configuration parameter shall be set to true for enabling the Safety Mechanisms implemented in the driver.

3.1 Range Check

All API parameters for configuration and control functions are checked in MCAL drivers according to ISO26262-6 table 4, 'Range checks of input and output data.

The AURIX™ SEooC ASW callout functions and additional Diagnostic Event manager (DEM) checks are implemented in MCAL drivers to report errors. This additional safety error reporting is configurable as either ON or OFF. Additional error list is further detailed at software module level in the respective MCAL module user manual.

[SM_AURIX_MCAL_021]The integrator shall activate the additional DEM checks by enabling <Mod>SafetyEnable configuration parameter and implement the callout function SafeMcal_ReportError() to handle the range check errors.[/req]

Each MCAL module has a configuration structure (<Mod>_ConfigType) and the pointer to the structure is passed to the Initialization function of the module. In order to validate the pointer, the structure is enhanced to add an additional 4 byte marker which can validate the pointer. This marker value holds ModuleID in the upper 16 bits and InstanceID in the lower 16 bits. The pointer is validated using the marker in the initialization function of each MCAL module.

For API's with pointer parameters, appropriate markers are added as configuration parameters and the same is validated within the API's of applicable MCAL modules. Refer the module specific user manuals for the parameter names in configuration.

For example, AdcGroupBufferMarker (configuration parameter for ADC) used for validating the pointer parameter in Adc_SetupResultBuffer.

[SM_AURIX_MCAL_022]The integrator shall write the configured buffer marker value to the first or last index of the passed buffer as per the requirement of a module.[/req]

Confidential

For example, if value of parameter AdcGroupBufferMarker is 0xC8, then the value 0xC8 should be written to the first index of the passed buffer i.e., ResultBuffer[0] = 0xC8.

[SM_AURIX_MCAL_098]Integrator shall ensure that the valid pointer is passed to <Mod>_GetVersionInfoApi() to avoid memory corruption, if wrong pointer is passed. [/req]

Note: Valid pointer means: no NULLPTR and a writeable location with appropriate access rights for the caller

Rationale: Pointer range check is not done for the <Mod>_GetVersionInfoApi()

3.2 Freedom from Interference with Respect to Memory

MCAL software can be partitioned between ASIL B and QM software by using different or independent hardware peripherals or kernel. This shall be ensured by configuration. Development of each driver is done at an assigned ASIL level.

The variables, SFR and code sections used by ASIL B MCAL drivers are published in generated file <mod>_ConfigDoc.html. This information is useful to the user to set the appropriate MPU settings to ensure proper freedom from interference against other non-safety drivers or drivers with different safety level.

<Mod> = Adc, Mcu, Port, Dio, Icu, Pwm, Gpt, Spi and Crc

Specific configuration parameters are available to denote whether a channel or group is used for safety usage or QM usage. For example, one SPI hardware unit can be used for safety and the other for QM. Different or independent variables and buffers are used for the safety usage and QM usage, if applicable for the MCAL module. The resource usage documentation (<Mod>_ConfigDoc.html) explains the address and SFR ranges to be used in safety related usage and the same can be used in MPU.

[SM_AURIX_MCAL_025]The integrator shall enable the MPU setting in hardware as per the memory address and SFR ranges mentioned in <Mod>_ConfigDoc.html.[/req]

[SM_AURIX_MCAL_099]The integrator shall protect the Stack memory from the corruption using MPU protection to avoid corruption of stack memory leading to wrong behavior. [/req]

[SM_AURIX_MCAL_100]The integrator shall not call <Mod>_Init() and <Mod>_DeInit() APIs from QM partition for drivers targeted to be used with ASIL claim.[/req]

3.3 Initialization Check

An additional API <Mod>_InitCheck() has been added to check the initialization done by the respective module. This API is enabled when <Mod>InitCheckApi is set to TRUE.

[SM_AURIX_MCAL_026]Initialization function of MCAL module initializes the fixed variables and fixed SFRs according to the configuration. Module initialization shall be called before any other API call of the module. In order to guarantee safe initialization, initialization check API shall be called to check the internal fixed variables and fixed SFRs, after initialization of all modules. Any control and read APIs shall be called only after module initialization check is completed. It is assumed that the SFRs and memory is protected from interference using MPU or equivalent measures by the integrator before invoking the Init Check.[/req]

Example

1. <Mod> Init() of all module
2. <Mod> Init Check() of all module
3. Control and Read API (can be more than one module)

Rationale: Module initialization check function verifies all fixed SFRs and fixed variables initialized by module initialization function against the configured value. It does not modify any SFR or variable but only a read operation is done. If any failure in comparison is found, it reports an error.

[SM_AURIX_MCAL_027]The integrator shall take an appropriate action, if error is reported by <Mod> Initialization check function.[/req]

[SM_AURIX_MCAL_028]In case of re-initialization (after de-init), the initialization check shall be executed by user software.[/req]

Confidential

Initialization check function shall be performed with the following steps on module re-initialization:

1. De-Init (optional due to re-initialization)
2. <Mod> Init (can be more than one module init)
3. <Mod> Init check (can be more than one module)

<Mod> = Adc, Mcu, Port, Dio (only for 4.0.3), Icu, Gpt, Spi, Pwm

Note: Pwm_InitCheck() API is not implemented in PWM driver. The integrator shall implement the check for initialization of PWM or use equivalent measures to detect any failures affecting the functionality (for example, corruption of SFRs, variables). The integrator may refer the initialization check example provided in PWM driver user manual section 9.3.

4 Assumptions of Use

This section provides the assumptions or requirements that need to be fulfilled by components outside of the AURIX™ SEooC ASW.

4.1 Configuration and Calibration

[SM_AURIX_MCAL_029] Tresos is the configuration tool used to generate data for safety related software modules and it shall be verified by integrator before usage. No safety mechanisms are implemented for the generation of configuration data. The integrator is responsible to verify the safety related configuration data generated from the configuration tool (Tresos). [/req]

The safety related MCAL software modules do not require any calibration data.

The integrator shall refer the release note of used MCAL software product version to know the supported version of Tresos.

4.2 Memory Protected Tasks

[SM_AURIX_MCAL_033] The AURIX™ hardware supports partitioning with a MPU. The integrator shall ensure freedom from interference, for example using the MPU in case of coexistence of ASIL and QM drivers. [/req]

Example: A QM driver shall not be part of an ASIL partition.

The MCAL safety drivers have a provision to separate signals by using different memory partitions (MPU based). This feature can be used for the argumentation of freedom from interference between ASIL and QM signals.

Example:

If ASIL and QM signals are to be acquired in ADC, integrator shall follow the below steps to achieve freedom from interference for MCAL between ASIL and QM signals

a. ASIL and QM signal shall be part of different ADC hardware kernel or cluster
b. ASIL signal channel's variables and SFR ranges shall be set up in a MPU partition
c. QM signal channel's variables and SFR ranges shall be set up in a different MPU partition
d. Common variables or SFR's between the ASIL and QM signals shall be set up in a MPU partition of ASIL signal

- *Task based protection is also dependent on OS implementation in the sense of how many regions per task are supported. Additionally, some of them will be used from the operating system internally. The limiting factor is therefore how many regions can be used to protect peripheral and internal variable accesses.*

4.2.1 Safety Mechanisms for Variables and SFRs

The following table lists the type of variables and how integrators safety concept can handle the same through MPU and initialization check functionality to prevent data corruption. The variables, SFR and code sections used by ASIL B MCAL drivers are published in generated file <Mod>_ConfigDoc.html.

<Mod> = Adc, Mcu, Port, Dio, Icu, Pwm, Gpt, Spi and Crc

Table 3 Safety mechanisms for different resource categories

Confidential

Global fixed variables		
	Description	Set by init and then not modified.
	Mechanisms	1. [SM_AURIX_MCAL_034]Only the init function writes these variables. Integrator shall have a dedicated data range setting in MPU for such variables to prevent any corruption. [/req] 2. Init check function independently checks if the initialization was correct.
Global non-fixed variable		
	Description	Changed after init
	Mechanisms	1. Init function writes a default value for these variables. 2. Init check function verifies if default value is correct. 3. [SM_AURIX_MCAL_035]The integrator shall monitor these variables to ensure they are correct (plausibility checks) or use MPU protection to prevent any corruption of these global non-fixed variable.[/req] *Usage of these variables in MCAL is avoided as far as possible.*
Global constants		
	Description	Always fixed; i.e. configuration (in Flash)
	Mechanisms	1. [SM_AURIX_MCAL_036]The integrator shall ensure the correctness of the global constants at system level by for example inspection/verification of values/contents and CRC.[/req] 2. Read-only data (ensured by hardware). • *The init-structure has an additional attribute (marker) to verify the pointer variable to the configuration structure as explained in **3.1***
Channel fixed variables		
	Description	Set by init and then not modified.
	Mechanisms	1. [SM_AURIX_MCAL_037]Only the init function writes these variables. The integrator shall ensure that QM and ASIL variables are placed in separate MPU partition to prevent any corruption.[/req] 2. Init check function independently checks the correctness of initialization.
Channel non-fixed variables		
	Description	Changed after init
	Mechanisms	1. Init function writes a default value for these variables. 2. Init check function verifies the written default value. 3. [SM_AURIX_MCAL_038]The integrator shall ensure that QM and ASIL variables are placed in separate MPU partition to prevent any corruption. [/req]
Global fixed SFRs		
	Description	Set by init and then not modified
	Mechanisms	1. [SM_AURIX_MCAL_040]Only init function writes these SFRs. The integrator shall have an MPU setting to prevent any corruption of the global fixed SFRs. [/req] 2. Init check function independently checks the correctness of initialization.
Global non-fixed SFRs		
	Description	Changed after init
	Mechanisms	Global non-fixed SFRs are not used in MCAL. Hence no safety mechanism is required to handle the global non-fixed SFRs.
Channel fixed SFRs		
	Description	Set by init and then not modified.
	Mechanisms	1. [SM_AURIX_MCAL_041]Only the init function writes these SFRs. The integrator shall have a MPU setting to prevent any corruption of the channel fixed SFRs. [/req] 2. Init check function independently checks the correctness of initialization.

Channel non-fixed SFRs	
Description	Changed after init • *SFRs for DMA channels are part of the Channel non-fixed SFRs.*
Mechanisms	1. Init function writes a default value for these SFRs. 2. Init check function verifies the written default value. 3. [SM_AURIX_MCAL_042]The integrator shall have a MPU setting to prevent any corruption of the channel non-fixed SFRs.[/req] 4. [SM_AURIX_MCAL_043]The integrator shall have two different MPU protections for ASIL and QM channels (For example, for ADC driver, ASIL and QM channels are separated based on ADC hardware groups). This is to enable QM and ASIL signals on the same ADC cluster.[/req]

4.3 Coexistence of QM and ASIL Signals

[SM_AURIX_MCAL_044]Freedom from Interference between ASIL and QM software shall be ensured from the integrator by using hardware supported protection mechanisms (MPU, restricted access to memory and SFR's).[/req]

[SM_AURIX_MCAL_045]The integrator shall assign QM and ASIL signals to different hardware peripherals or kernels. For example, in case of ICU, CCU6 unit or GTM TIM0 can be dedicated for QM signal and GTM TIM1 for ASIL signal. In case of port pin usage (DIO driver), pins used for ASIL related usage and QM related usage must not be mixed in the same PORT.[/req]

[SM_AURIX_MCAL_046]Different interrupts or interrupt sources shall be allocated for ASIL and QM channels or signals.[/req]

[SM_AURIX_MCAL_047]In case signal groups are required for application, the groups shall contain signals with the same ASIL level. i.e., all the QM signals shall be assigned to one hardware peripheral or kernel and all ASIL signals shall be assigned to other hardware peripheral or kernel. This shall be ensured by configuration.[/req]

[SM_AURIX_MCAL_051]SFRs and fixed variables that can generate common cause failures between ASIL and QM signals shall be protected. [/req]

[SM_AURIX_MCAL_052]The hardware functionality of bus access protection (ACCEN) shall be used to ensure freedom from interference of software from different bus masters (QM and ASIL). [/req]

- *This means that the module registers for access protection shall be controlled from integrator software and not from AURIX™ SEooC ASW software modules.*

4.4 Task Level Control Flow Monitoring

The logical and temporal program flow monitoring is not in the scope of MCAL software.

[SM_AURIX_MCAL_053]Control Flow Monitoring is expected to be implemented on application SW level (before and after calling the MCAL driver functions), and not within the MCAL drivers themselves.[/req]

4.5 Error Handling

[SM_AURIX_MCAL_054]The AURIX™ SEooC ASW safety-related callout functions and production error checks (like Diagnostic Event Manager) shall be handled within diagnostic time interval of the application specification. [/req]

Example:

- In the callout provided, application software could implement specific DEM errors for each module or a common DEM Safety error. Then application software could initiate fault logging followed by Reset.

[SM_AURIX_MCAL_055]MCAL modules only report the detected errors by callout and do not handle any of the detected errors. The integrator shall handle the reported error from MCAL to enter the safe state as per the safety concept of application.[/req]

Confidential

Rationale: MCAL is intended to use in AUTOSAR infrastructure where DEM is provided as central error handling module.

[SM_AURIX_MCAL_056]The integrator shall implement Timeout monitoring to detect permanent and transient hardware faults (such as “no interrupt”) in the interrupt router module. Safety Analysis shall be performed by user to identify possible undetected malfunctions due to permanent and transient hardware faults. [/req]

5 Module Specific Safety Information and AoU’s

This section describes the module specific safety information and the module specific AoU’s. Safety mechanisms and AoU’s common for safety drivers are mentioned in chapter **3** and chapter **4.**

The integrator shall refer the user manual of MCAL modules to know the APIs supported in safety use case and the reported errors.

5.1 MCU Driver

[SM_AURIX_MCAL_057]The integrator shall call the Mcu_InitCheck() API after the execution of initialization functions.[/req]

Example:

a. Call Mcu_Init() API
b. Call Mcu_InitClock() API
c. Call Mcu_GetPllStatus() API
d. Call Mcu_DistributePllClock() API
e. Call Mcu_InitCheck() API

Rationale: APIs Mcu_Init(), Mcu_InitClock(),Mcu_GetPllStatus() and Mcu_DistributePllClock() are required for proper initialization of MCU driver, hence it is recommended to call Mcu_InitCheck() API after calling all these APIs.

[SM_AURIX_MCAL_101]The integrator shall not call the following APIs from QM partition, if MCU driver is targeted to be used with ASIL claim.

a. Mcu_InitClock()
b. Mcu_DistributePllClock()
c. Mcu_RampToBackUpClockFreq()
d. Mcu_17_CcuconRegUpdate()
e. Mcu_SetMode()
f. Mcu_SetStandbyCtrlReg()[/req]

[SM_AURIX_MCAL_102]The integrator shall provide access to Mcu_Init() API for all the resources used by GTM complex driver, if used in the system for the GTM complex driver initialization including the required SFRs. [/req]

Rationale: Mcu_Init() API performs the GTM complex driver initialization including the required SFRs.

[SM_AURIX_MCAL_058]The integrator shall call Mcal_ResetSafetyENDINIT_Timed() before calling Mcu_PerformReset() API to perform microcontroller reset. The library call Mcal_ResetSafetyENDINIT_Timed() disable interrupts; hence no need of explicitly disabling interrupts to ensure no pre-emption happens.[/req]

Rationale: This is to provide a safety mechanism to prevent unintended call of Mcu_PerformReset(). Enabling of write protection for the reset register is done by invoking the API Mcal_ResetSafetyENDINIT_Timed().

[SM_AURIX_MCAL_059]The integrator shall not call Mcu_PerformReset() API from lower level ASIL context.[/req]

Rationale: This is required for system in which an unintended reset violates a safety goal up to ASIL B.

Confidential

Note: Mcu_PerformReset() can be called from QM context in systems where reset is considered as a safe state. The QM task shall be responsible to enable write protection of the reset register before invoking Mcu_PerformReset().

Example:

The integrator shall call the APIs in following sequence to perform reset.

a. Call Mcal_ResetSafetyENDINIT_Timed()
b. Call Mcu_PerformReset()

[SM_AURIX_MCAL_060]The integrator shall call Mcal_ResetENDINIT() or Mcal_ResetSafetyENDINIT_Timed() before calling Mcu_SetMode() API and call Mcal_SetENDINIT() or Mcal_SetSafetyENDINIT_Timed() after calling Mcu_SetMode() to perform power down. The library call Mcal_ResetENDINIT() or Mcal_ResetSafetyENDINIT_Timed() disable interrupts, hence no need of explicitly disabling interrupts to ensure no pre-emption happens.[/req]

Rationale: This is to provide a safety mechanism to prevent unintended call of Mcu_SetMode(). Enabling of write protection for the power down register is done by invoking the API Mcal_ResetENDINIT() or Mcal_ResetSafetyENDINIT_Timed().

[SM_AURIX_MCAL_061]The integrator shall invoke an additional Mcu_GetMode() API from another core to verify if the intended core has gone to IDLE state. It cannot verify SLEEP or STANDBY state since the entire system goes to power down state. This measure is applicable for controllers with multi core only

For SLEEP or STANDBY modes, user shall start a timer with notification before calling Mcu_SetMode(), such that the timer expires and provides notification, if system has not entered SLEEP or STANDBY mode.[/req]

Example:

The integrator shall call the APIs in following sequence to achieve safe power down to IDLE state.

a. Call Mcal_ResetENDINIT()
b. Call Mcu_SetMode()
c. Call Mcal_SetENDINIT()
d. Call Mcu_GetMode() in different core

Rationale: The desired power down mode may not be triggered by Mcu_SetMode() API due to software faults.

[SM_AURIX_MCAL_062]The integrator shall not call Mcu_SetMode() API from lower level ASIL context.[/req]

Rationale: This is required for the system in which an unintended power down violates a safety goal up to ASIL B.

Note: Mcu_SetMode() can be called from QM context in systems where power down is considered as a safe state. The QM task shall be responsible to enable write protection of the power down register before invoking Mcu_SetMode().

[SM_AURIX_MCAL_063]The integrator shall call Mcal_ResetSafetyENDINIT_Timed() before calling Mcu_ClrWkpStdby() or Mcu_SetStandbyCtrlReg() API to perform power down and clear the status of wakeup. The library call Mcal_ResetSafetyENDINIT_Timed() disable interrupts, hence there is no need of explicitly disabling interrupts to ensure no pre-emption happens.[/req]

Rationale: This is to provide a safety mechanism to prevent unintended call of Mcu_ClrWkpStdby() and Mcu_SetStandbyCtrlReg(). Enabling of write protection for the power down register is done by invoking the API Mcal_ResetSafetyENDINIT_Timed().

Example:

The integrator shall call the APIs in following sequence to achieve safe power down to SLEEP/STANDBY state

a. Call Mcu_ChkWkpStdby() in user Task 1

b. Call Mcal_ResetSafetyENDINIT_Timed() in user Task 1
c. Call Mcu_ClrWkpStdby() in user Task 1
d. Call Mcal_SetSafetyENDINIT_Timed() in user Task 1
e. Call Mcu_RampToBackUpClockFreq() in user Task 1
f. Start GPT timer with notification in user Task 2 (Refer the GPT module user manual for the exact API name)
g. Call Mcal_ResetENDINIT() in user Task 1
h. Call Mcal_ResetSafetyENDINIT_Timed() in user Task 1
i. Call Mcu_SetMode() in user Task 1
j. Call Mcal_SetSafetyENDINIT_Timed() in user Task 1
k. Call Mcal_SetENDINIT() in user Task 1

[SM_AURIX_MCAL_065]The integrator shall configure and enable clock monitoring for the desired clocks by enabling the configuration parameter McuClockMonitoringEnable.[/req]

Rationale: To ensure the correct operation of the clock system and the desired clock frequencies are generated, the hardware safety mechanism clock monitoring shall be used.

[SM_AURIX_MCAL_066]The integrator shall enable (configure) the SMU to raise an alarm, if the clock monitoring unit reports an error for the clock frequencies which are not in proper range.[/req]

Rationale: MCAL driver only enables the hardware clock monitoring feature and the detected error by hardware clock monitoring unit is not identified by MCAL driver.

5.2 WDG Driver

[SM_AURIX_MCAL_067]The WDG Driver is based on the functional requirements from AUTOSAR. However, the WDG driver is enhanced (with respect to AUTOSAR functionality) to use the password mechanism provided in the CPU watchdog hardware to realize the functionality of program flow monitoring. The driver requirements are developed according to a QM process assuming the integrator performs ASIL decomposition at the system level (for example, using concepts like black channel). Subsequently, these drivers can be used in a safety relevant application.

The WDG driver is part of the black channel (QM) and the associated safety mechanism, for example. safety checks (password verification) is performed by the CPU watchdog hardware.[/req]

Note: Usage of the password mechanism in CPU watchdog hardware also needs an enhanced WDG Manager and an enhanced WdgIf module.

Confidential

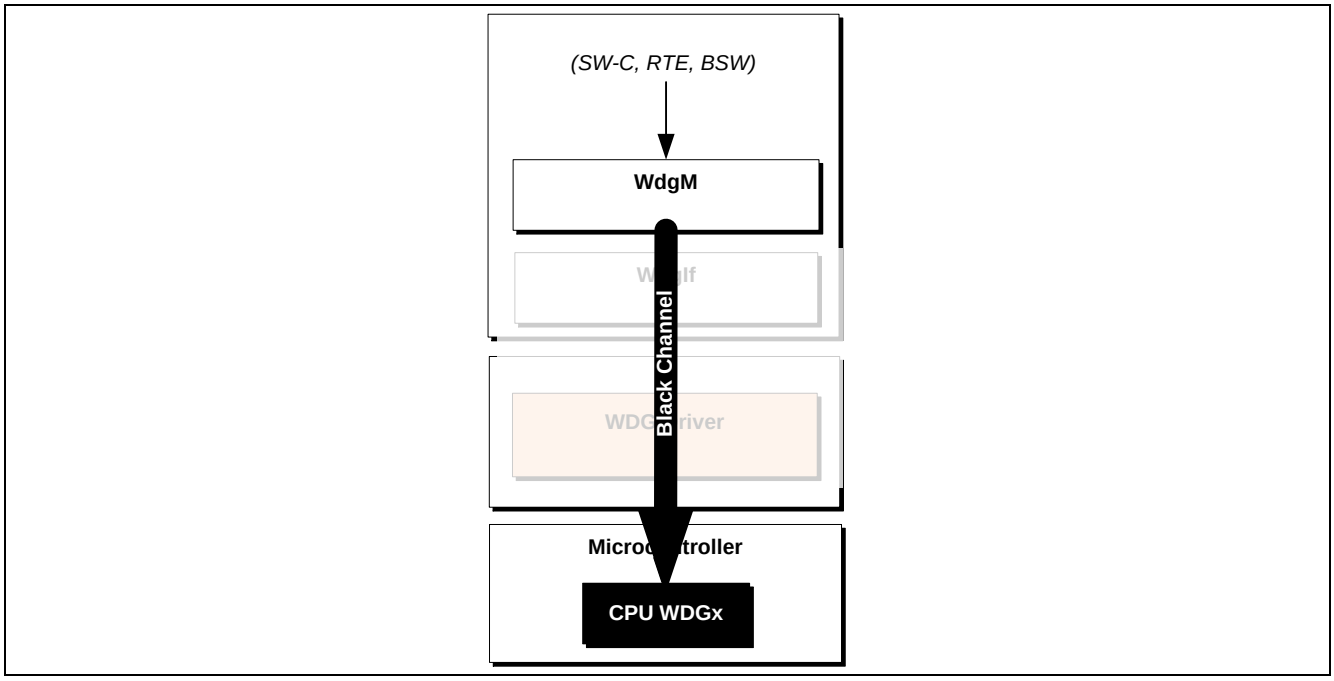

Figure 2 Black Channel for WDG driver

[SM_AURIX_MCAL_069]WDG shall be initialized either in SLOW/FAST mode and shall not be initialized in OFF mode.[/req]

Rationale: WDG OFF mode disables the WDG peripheral and the Safety features.

5.3 GPT Driver

[SM_AURIX_MCAL_070]PWM driver Initialization shall be performed before any call to Gpt_StartTimer() or Gpt_StopTimer() APIs. This is also in line with AUTOSAR requirement to perform all initialization function of the driver together (through EcuM) before using the control API's.[/req]

Rationale: This is to avoid any unexpected start or stop of the GPT timers due to the use of common TOM/ATOM registers between PWM and GPT drivers.

[SM_AURIX_MCAL_071]The integrator shall limit one TOM/ATOM module to one driver, for instance TOM0 shall be allocated to PWM and TOM1 shall be allocated to GPT.[/req]

Rationale: This avoids common cause fault due to common TOM/ATOM registers (for example, TGC register).

[SM_AURIX_MCAL_072]The integrator shall ensure the passing of correct parameter to Gpt_CheckWakeup() from EcuM as it is not possible to have range check within the GPT driver.[/req]

[SM_AURIX_MCAL_073]The integrator shall call Gpt_GetTimeElapsed() or Gpt_GetTimeRemaining() API at least twice at a minimum threshold (one GPT timer ticks) difference to cross verify the timer status in polling mode.[/req]

Rationale: This is required to identify the stuck at fault due to software and hardware.

[SM_AURIX_MCAL_074]The integrator shall call Gpt_GetTimeElapsed() API only after Gpt_StartTimer() API, to ensure that Gpt_StartTimer() is executed correctly.[/req]

[SM_AURIX_MCAL_075]The integrator shall call Gpt_GetMode() API after Gpt_SetMode() API to confirm the correct update of GPT driver mode.[/req]

[SM_AURIX_MCAL_103]The integrator shall not call Gpt_SetMode() API from QM partition, if GPT driver is targeted to be used with ASIL claim.[/req]

[SM_AURIX_MCAL_104]The integrator shall assign QM and ASIL GPT channels to different TOM/ATOM module.[/req]

Rationale: This is required to ensure freedom of interference between QM and ASIL channels.

Confidential

5.4 ICU Driver

[SM_AURIX_MCAL_076]The integrator shall ensure the passing of correct parameter for Icu_17_GtmCcu6_CheckWakeup() API from EcuM as it is not possible to have range check within the ICU driver. [/req]

[SM_AURIX_MCAL_077]The integrator shall not use the ERU signals in a safety use case, since the ICU driver treats the signals mapped to ERU hardware as a QM signal.[/req]

Rationale: All supported ICU modes are not possible to realize using ERU hardware. Hence, ICU edge detection using ERU hardware is supported only in QM use case.

[SM_AURIX_MCAL_105]The integrator shall not call Icu_17_GtmCcu6_SetMode() API from QM partition, if ICU driver is targeted to be used with ASIL claim. [/req]

[SM_AURIX_MCAL_106]The integrator shall not use the Wakeup functionality in QM partition, if ICU driver is targeted to be used with ASIL claim. [/req]

[SM_AURIX_MCAL_107]The integrator shall configure the different CCU6 module for ASIL and QM usage, if ICU driver is targeted to be used with ASIL claim. [/req]

[SM_AURIX_MCAL_079]The integrator shall write the configured buffer marker value to the first index or address of the passed buffer.[/req]

For example, if value of parameter IcuDutycycleBufferMarker is 0xC8, then the value 0xC8 shall be written to the first index or address of the pointer passed to API Icu_17_GtmCcu6_GetDutyCycleValues() i.e., DutyCycleValue[0] = 0x000000C8.

Rationale: Pointers given to the MCAL have to fulfill ASIL B integrity level for ASIL B modules.

5.5 SPI Driver

[SM_AURIX_MCAL_080]The integrator shall use end-to-end safe communication protocol, whenever possible and if the connected SPI device supports this.[/req]

[SM_AURIX_MCAL_081]The integrator may disable the safety mechanisms implemented in SPI driver by setting the configuration parameter SpiSafetyEnable to false, when end-to-end safe communication protocol is used by the application for the SPI communication.[/req]

Rationale: No safety measures are required for SPI Handler driver, if end-to-end safe communication protocol is used.

[SM_AURIX_MCAL_082]The integrator must not use the POLLING mode for usage of SPI driver in safety application.[/req]

Rationale: This is due to the limitation of SPI Main Function given in AUTOSAR. The function is applicable for the whole driver and not per HW Unit.

[SM_AURIX_MCAL_083]The integrator shall call Spi_SetAsyncMode() API to set asynchronous transmission to INTERRUPT mode, after Spi_Init() and before calling Spi_AsyncTransmit().[/req]

Example:

a. Call Spi_Init()
b. Call Spi_SetAsyncMode()
c. Call Spi_AsyncTransmit()

Rationale: In SPI level 2 configurations, POLLING mode is set by default for asynchronous transmission and POLLING mode is not supported for ASIL usage. Hence, the mode shall be changed from POLLING to INTERRUPT using Spi_SetAsyncMode() API.

[SM_AURIX_MCAL_085]The integrator shall ensure the QSPI kernels and DMA channels are not shared between QM and ASIL sequences.[/req]

Rationale: This is required to ensure freedom from interference between QM and ASIL jobs.

Confidential

[SM_AURIX_MCAL_108]The integrator shall not call the following APIs from QM partition, if SPI driver is targeted to be used with ASIL claim.

a. Spi_SetAsyncMode()
b. Spi_AsyncTransmit()
c. Spi_SyncTransmit()
d. Spi_WriteIB()
e. Spi_SetupEB()
f. Spi_IsrQspiError()
g. Spi_IsrQspiPt()
h. Spi_IsrDMAQspiRx()
i. Spi_IsrDmaError()
j. Spi_IsrDmaQspiTxRuntime()
k. Spi_IsrQspiUsr() [/req]

[SM_AURIX_MCAL_086]The integrator shall write configured unique buffer marker value with an additional byte at the end of the external buffers passed to SPI driver APIs.[/req]

Example:

1. For a 8-bit external buffer channel with 8bytes as transfer length and value of parameter SpiSafetyBufferMarker is 0xC8, integrator shall place the buffer marker at the 9th byte (for example, SrcBuffer[9]=0xC8) for both source and destination external buffers.

2. For a 16-bit external buffer channel with 8-bytes as transfer length and value of parameter SpiSafetyBufferMarker is 0xC8, integrator shall place the buffer marker at the 9th and 10th bytes (for example, SrcBuffer[4]=0x00C8) for both source and destination external buffers.

Rationale: Pointers given to the MCAL have to fulfill ASIL B integrity level for ASIL B modules.

5.6 ADC Driver

[SM_AURIX_MCAL_087]The integrator must not use Background source, EMUX (External Multiplexer) Scan and Non-AUTOSAR DMA result handling features for ASIL safety related usage.[/req]

Rationale: It is not possible to independently start/stop the background source for QM and ASIL signals, since there is only one background source for all the ADC HW groups.

[SM_AURIX_MCAL_088]The integrator shall write the configured unique buffer marker value to the first index or address of the buffer passed to ADC driver APIs.[/req]

For example,

1. if the datatype of buffer is 8-bit and the value of parameter AdcGroupBufferMarker is 0xC8, then the value 0xC8 shall be written to the first index or address of the passed buffer i.e., ResultBuffer[0] = 0xC8.

2. if the datatype of buffer is 16-bit and the value of parameter AdcGroupBufferMarker is 0xC8, then the value 0x00C8 shall be written to the first index or address of the passed buffer i.e., ResultBuffer[0] = 0x00C8.

Rationale: Pointers given to the MCAL have to fulfill ASIL B integrity level for ASIL B modules.

[SM_AURIX_MCAL_109]The integrator shall ensure that buffer provided is properly aligned (16bit memory alignment required, in case 10bit or 12bit resolution is used for any channel).[/req]

[SM_AURIX_MCAL_110]The integrator shall read the data only after confirming the channel specific conversion is completed in polling mode using Adc_17_GetChannelStatus() API.[/req]

[SM_AURIX_MCAL_089]The integrator shall assign QM and ASIL signals (software groups) to different ADC hardware groups.[/req]

Confidential

Rationale: This is required to ensure freedom from interference between QM and ASIL signals.

[SM_AURIX_MCAL_111]The integrator shall configure the different pairs of ERU units to ASIL and QM ADC hardware groups.

For example, ERU0 and ERU1 can be assigned to ASIL ADC hardware groups, ERU2 and ERU3 can be assigned to QM ADC hardware groups.[/req]

Rationale: This is required to avoid the common cause failure, since two ERU units shares the same SFR

5.7 PWM Driver

[SM_AURIX_MCAL_091]The integrator shall verify the correctness of initialization done by the Pwm_17_Gtm_Init() API, after initialization of all modules to ensure no corruption of variables and SFR. Alternatively, verify the generated PWM signals using redundancy (for example, feedback or loopback mechanisms using ICU driver) to detect the errors in the generated PWM signal.[/req]

[SM_AURIX_MCAL_092]The integrator shall limit one TOM/ATOM module to one driver, for instance TOM0 shall be allocated to PWM and TOM1 shall be allocated to GPT.[/req]

Rationale: This avoids common cause fault due to common TOM/ATOM registers (e.g. TGC register). The TGC registers is common per 8 channels in a TOM module. This register cannot be initialized in GTM Initialization (part of MCU) due to functional issue as only part of the register is common and the rest is specific to the type of configuration used, however the initialization has to be done in one access for a proper run.

[SM_AURIX_MCAL_093]The integrator shall write configured unique buffer marker value with an additional byte at the last of the external buffers passed to PWM driver. The marker shall be same as the configuration value of parameter PWMDutycycleBufferMarker. It is recommended that the value configured is not to be a valid DUTY cycle value.[/req]

Example: If value of parameter PWMDutycycleBufferMarker is 0xC8 for a 8 channel PWM synchronized group, user shall place the buffer marker at the 9th byte (for example, SyncDuty[9]=0x000000C8) and pass it API Pwm_17_Gtm_SyncGrpUpdateDuties().

Rationale: Pointers given to the MCAL have to fulfill ASIL B integrity level for ASIL B modules.

[SM_AURIX_MCAL_094]The integrator shall assign QM and ASIL PWM channels to different TOM/ATOM module.[/req]

Rationale: This is required to ensure freedom of interference between QM and ASIL channels.

5.8 PORT Driver

[SM_AURIX_MCAL_112]The integrator shall not call the Port_RefreshPortDirection() API from QM partition, if PORT driver is targeted to be used with ASIL claim.[/req]

[SM_AURIX_MCAL_113]The integrator shall call Port_SetPinDirection() and Port_SetPinMode() APIs from QM and ASIL partition, only for the pins of a different port.[/req]

5.9 DIO Driver

[SM_AURIX_MCAL_114]The integrator shall call Dio_WriteChannelGroup() API from QM and ASIL partition, only for the channel groups of a different port.[/req]

5.10 CRC Library

No module specific AoUs for CRC library usage.

5.11 MCAL Library

Certain common routines are available within MCAL (accessed between QM and ASIL drivers) and SafeTlib. These routines can be classified into two groups:

WDG EndInit Routines: These routines called by MCAL drivers either in the initialization routines or in de-initialization routines and these common routines are allocated at ASIL B. There is no risk if a QM driver or ASIL

Confidential

B driver call these routines. Global variables used in these functions are implemented with diverse storage to prevent any corruption. In case of failure a call back function named Mcal_SafeErrorHandler is called.

[SM_AURIX_MCAL_095]The integrator shall implement the function Mcal_SafeErrorHandler() in application to handle the error reported by WDG EndInit routines.[/req]

DMA routines and certain core specific routines: DMA routines are called from ADC or SPI. They are primarily inline functions and no global variables are used in these routines. The DMA channels that are used are independent per usage of the module, i.e. if SPI hardware unit 0 uses DMA channel 0 for reception, this will not be changed within the application life time, hence the MPU protection of caller module is sufficient to ensure the freedom from interference between different accesses.

[SM_AURIX_MCAL_115]The integrator shall ensure that valid values are passed to all MCALLIB APIs as per the input parameter range, if these functions are called directly in applications.[/req]

[SM_AURIX_MCAL_116]The integrator shall ensure that the APIs Mcal_SafetyModifyAccess() and Mcal_SafetyCheckAccess() are not

1. Called concurrently from other cores
2. Called when Mcal_ResetSafetyENDINIT_Timed()/Mcal_SetSafetyENDINIT_Timed() are being executed or vice versa[/req]

Rationale: Simultaneous access to the WDTCON registers may occur leading to unexpected behaviour of the safety watchdog. (e.g., Wrong password leading to alarms, wrong time setup, etc)

6 AoU's of AURIX Hardware Safety Manual Implemented in MCAL

Table 4 List of AoU's Implemented in MCAL

AoU ID	Comments/Implementation hints from MCAL
SM1[AoU].IR:SrcCheck	Interrupt source check is implemented in MCAL for the ASIL drivers
SM_AURIX_QSPI_1 SM_AURIX_ADC_1 SM_AURIX_DIO_1	Support for clock monitoring is provided in MCU driver. The integrator shall configure and enable clock monitoring for the desired clocks by enabling the configuration parameter McuClockMonitoringEnable.

Note: AoU's of AURIX Hardware Safety Manual other than mentioned in **Table 4** *are not implemented in MCAL and requires evaluation of the integrator.*

.

w w w . i n f i n e o n . c o m

Published by Infineon Technologies AG